STAGE 2 – PRACTICAL ENGINEERING CHALLENGE

TEAM ID:	TEAM NAME:
LRC – 26 – 0642	EnergiX

Problem Statement: Industrial Manufacturing Process Automations

Industrial manufacturing depends on highly controlled liquid-handling systems to deliver process fluids reliably from a source or bulk supply to downstream process equipment. Depending on the application, these fluids may include **ultrapure fluids, process chemicals, solvents, developers, or CMP-related fluids**. Industry standards explicitly address liquid-chemical distribution, high-purity piping, bulk chemical distribution, system qualification, and UPW system operation and monitoring.

In such systems, **flow is a fundamental process variable**. An incorrect flow rate can indicate or cause a restriction, incorrect valve state, inadequate pumping performance, abnormal process demand, or another hydraulic deviation. The consequence may be unstable process conditions, incorrect chemical delivery, equipment malfunction, or loss of process performance.

This challenge therefore focuses on a fundamental industrial facilities and process-engineering task: **To establish a defined process-fluid flow, measure it reliably, detect a deviation, diagnose its cause, and restore the process to the required operating condition.**

Engineering Challenge

As you are part of a multidisciplinary engineering team responsible for designing, building and validating a small-scale automated liquid-flow system represented of industrial process handling system.

CASE 1 – CONTROL + MEASUREMNT

Problem: Develop a system that manages the changing level of fluid in the sump automatically (when there is a sudden external fluid charging to the sump) and while accurately measuring and totalizing the fluid discharged from the sump using the flow transmitter.

CASE 2 – FLUSHING +SHUTDOWN

Problem: Develop an alternative operating configuration that allows flush water to pass through the system while all three pumps operate simultaneously, and then safely transition the system to an OFF condition when the flush-water supply has been exhausted.

Given Process Parameters

Pipe diameter =	8mm
Fluid density (ρ) =	1000kg/m^3
Dynamic Viscosity (μ) =	1x10^{-3}Pa.s
Required Flow range =	1-6 L/min
Nominal target flow for the live challenge =	3L/min

Available Hardware

Mechanical Hardware	Electronic Hardware
1. Water Tank – 3 Quantity	1. Custom PCB
2. Pumps – 3 Quantity	2. ESP32 NodeMCU module
3. FT-01 Flow Transmitter – 01 Quantity	3. Pump Driver-1
4. Ball valve – 3 Quantity	4. Pump Driver-2
5. ON/OFF/XV Valve – 1 Quantity	5. Pump Driver-3
6. 10-mm OD tubing/blue – 9 meters/1 meter	6. 12 V DC power input (SMPS)
7. Microcontroller/PCB/ data-acquisition software	7. 12 VDC Solenoid-valve interface
8. Required electrical/ electronic accessories	8. Flow-sensor interface
	9. Float-switch interface
	10. ON/OFF-switch interface
	11. Pump-driver PWM/enable interfaces
	12. On/Off Switch

The team must determine the appropriate arrangement, connections and measurement method.

- Assemble and commission the liquid-flow system using the components provided.
- Measure and characterize the flow using FT-01
- Interface FT-01 with the provided **data-acquisition system**
- Experimentally determine the relationship between sensor output and actual flow.
- Characterize the effect of valve position on flow rate. Record and analyse flow vs time data
- Maintain target flow of 3L/min
- Use graphs and measured data to justify engineering decisions
- Design an appropriate hydraulic arrangement and P&ID

TEAM INSTRUCTIONS

- As a team go through the list provided to read about each component that has been Listed.
- The PCB is a 12 V DC and the SMPS connections should be made carefully.

TASK 1: HYDRAULICS SYSTEM DESIGN

Q1.1 Design and build the hydraulic path with the flow transmitter on the recirculation line. Submit/sketch a labelled P&ID below by showing the pumps, Needle valves, flow transmitter, flow direction, Tank, sump, tube diameter and measurement points.

CASE 1 – Process Fluid Transfer & Automatic Level Protection

Objective: Set-up and operate a closed-loop process- fluid handling system using the available tank, pump, valves flow transmitter, sump, float sensor and control PCB. The System must continuously transfer process fluid recirculation system by connecting the 10L fluid tank with a On/Off valve to a pump-01 and the pump-01 goes to the inlet of sump 4.2L tank. The Sump 4.2L outlet line is connected to another pump-02 followed by a flow transmitter that measures the total flow back to the 10L fluid tank. The Pump-01 runs on 50% duty cycle and Pump-02 runs on 100% duty cycle. **Calibrate the flow transmitter** and demonstrate correct fluid circulation, if there is sudden introduction of external fluid into the Sump tank then Pump-01 should stop while pump – 02 should still runs till fluid level reduces in the sump. After the fluid reduction in the SUMP 4.2L tank the Pump-01 should turn on and the process must continue. **Also, demonstrate Flow vs Time trends in mL/s, total transferred Liters back to the 10L tank** (share Screenshots flow in L/min). **Share the code.** The case-1 can be turned on using the available on/off Switch, to start the system.

CASE 2 – Flush/Line Conditioning Sequence

Objective: After successfully completing case 1, initiate the flush water in the sequence using the **equal Tee.** The On/Off valve must be switched off in order to isolate the 10L process fluid tank. The flush Fluid tank (4.2L) is used as a temporary fluid source to flush out the process fluid or any debris from the system. Before the start of this process make sure you open the manual ball valve on the fluid flush line before Pump-03 starts. The entire process of case -1 and case - 2 should run in a single go without manual intervention except opening of ball valve before the Pump-03 starts. **The System must stop as soon as the tank flush fluid 4.2L tank finishes.** The system must be automated without human interventions (only ball valve can be opened manually before the start of flush line sequence)

SUBMIT:

A. Submit: 10-15 min Video and Photograph of the completed setup, with identification of each component and explain the direction of flow in the video verbally. Demonstrate both Case - 1 and Case – 2.

B. Final P&ID Block diagram

RAID SKETCH / WORKING SPACE

Case 1 - process fluid Transfer & Automatic level protection

The hydralic system consists of a main 30L main process tank, an on/OFF solenoid valve, pump 1, a 4.2 L sump tank with float switch, pump-2 & the FT-01 flow transmitter

flow path: 30L main tank → ON/OFF valve (ov-01) → Pump-1 (50% duty cycle) → 4.2L sump tank with float switch, pump 2 & FT-01 flow transmitter (100% duty cycle) → FT-01 flow transmitter → 30L main tank.

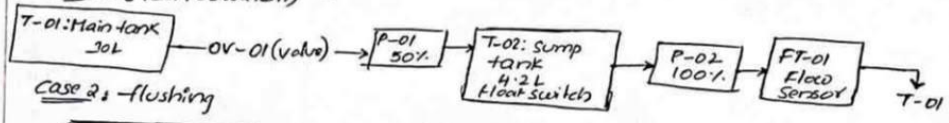
Working principle:

Pump 1 transfers water from main tank to sump at 50% duty cycle pump 2 returns water to main tank at volume in return line FT-01 measures flow rate & total volume in return line. If sump level becomes hight, the float switch signals the ESP32 to stop pump-1. Pump-2 continues until the sump level drops, Pump-1 restarts.

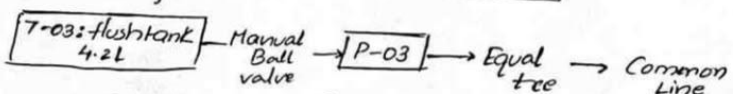
Case 2: FLUSH & SHUTDOWN

The 4.2L flush tank supplies water through pump-3 & an equal tee into common line. Close ov-01 to isolate the main tank & open the manual ball valve before starting. Run all three pumps during flushing. when flush water is exhausted, the controller stops all pump & switches the system OFF.

Case 1: (recirculation)



Case 2: flushing



P-01 + P-02 + P-03 (run simultaneously) Flush tank empty (All pumps OFF)

Q1.2 After setting-up the system, now operate it at approximately at 1L/min, 3L/min & 6L/min. Compare the theoretical velocity with the actual operating behaviour.

WORKING / COMPARISON

Given, pipe diameter $d = 8mm = 0.008m$

required flow rates: 1, 3, 6 L/min

Cross-sectional area of pipe: $A = \frac{\pi d^2}{4}$

$A = \frac{\pi(0.008)^2}{4} = 5.027 \times 10^{-5} m^2$

theoretical fluid velocity is calculated using: $v = \frac{Q}{A}$

Converting flow rate:

1 L/min $\Rightarrow 1.667 \times 10^{-5} \Rightarrow 0.332$ m/s

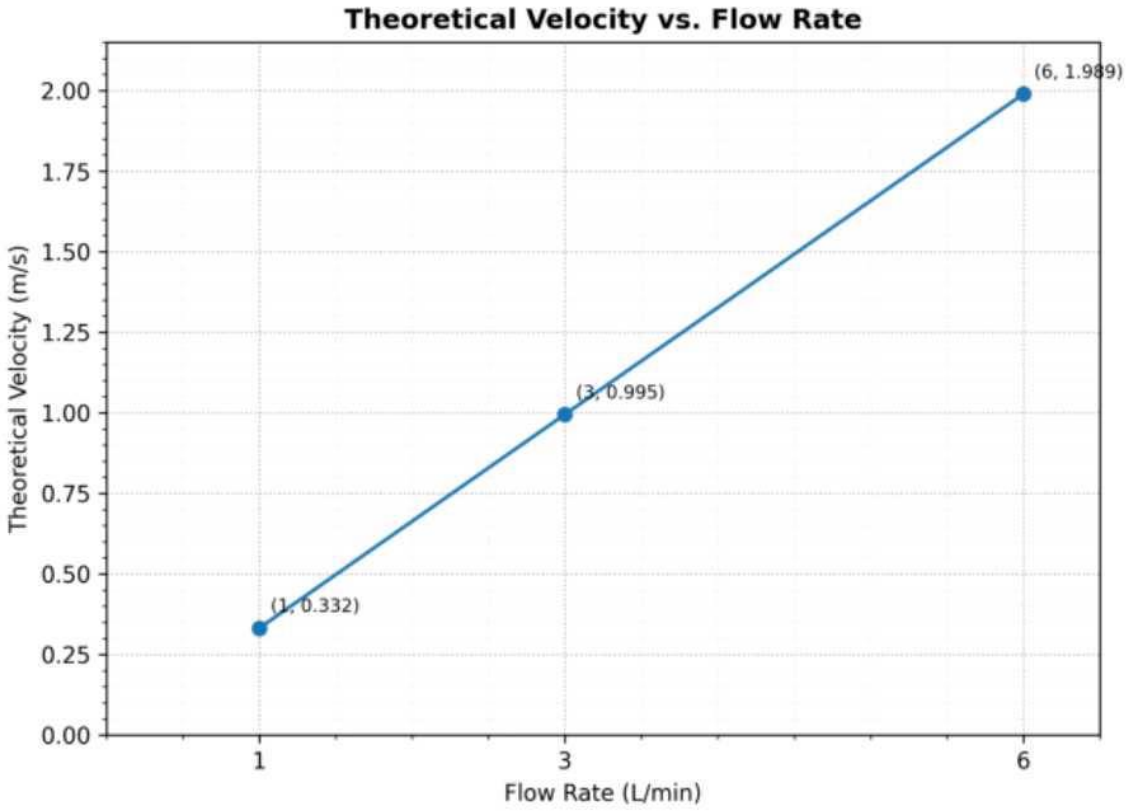
3 L/min $\Rightarrow 5.000 \times 10^{-5} \Rightarrow 0.995$ m/s

6 L/min $\Rightarrow 1.000 \times 10^{-4} \Rightarrow 1.99$ m/s

Comparision:

As the volumetric flow rate increases, the average fluid velocity also increases approximately linearly, provided the pipe diameter remains constant. Actual operating behaviour may differ from theoritical values due to pump performance, valve restrictions, friction losses, air bubbles & flow-sensor calibration errors. FFOI readings should be recorded after the flow stabilises.

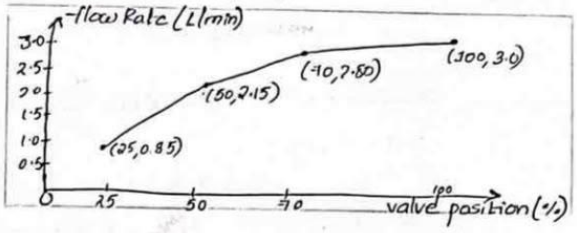
GRAPH:


Theoretical Velocity vs. Flow Rate
Theoretical Velocity (m/s)
Flow Rate (L/min)
0.00
0.25
0.50
0.75
1.00
1.25
1.50
1.75
2.00
1
3
6
(1, 0.332)
(3, 0.995)
(6, 1.989)

TASK 2: FLOW RESPONSE TO VALVE POSITION

Q 2.1 Set the ball valve to approximately 25%,50%,75% & 100%. At each position, allow the flow to stabilize and record the flow rate. Create the following graph of valve position (%) vs Flow Rate (L/min)

VALVE POSITION	FLOW RATE
25%	0.85
50%	2.15
70%	2.80
100%	3.00



Q2.2 Determine the experimental calibration equation: $Q = af + b$, where Q = flow rate in L/min, f = pulse frequency in Hz, a = experimental calibration coefficient and b = offset.

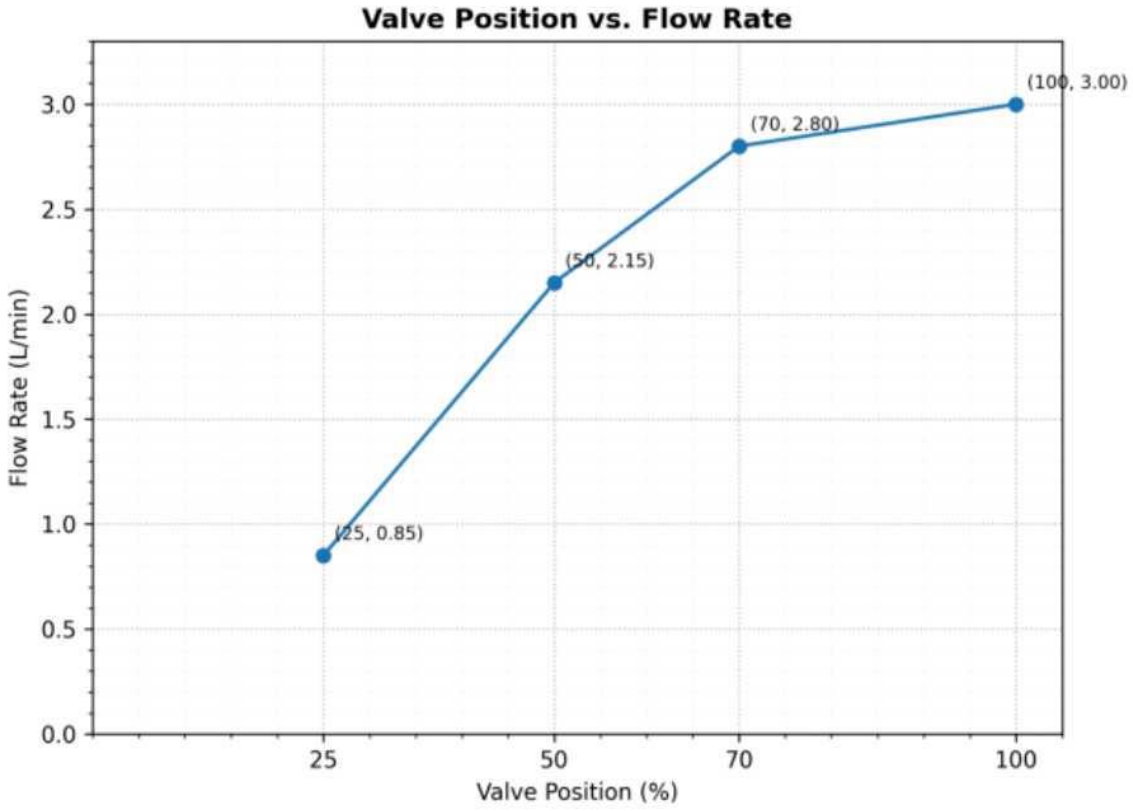
Report:

R^2 =	0.988
a =	0.133 L/(min·Hz)
b =	0.05 L/min
Maximum calibration error =	±4.2%
Average calibration error =	± 1.5%

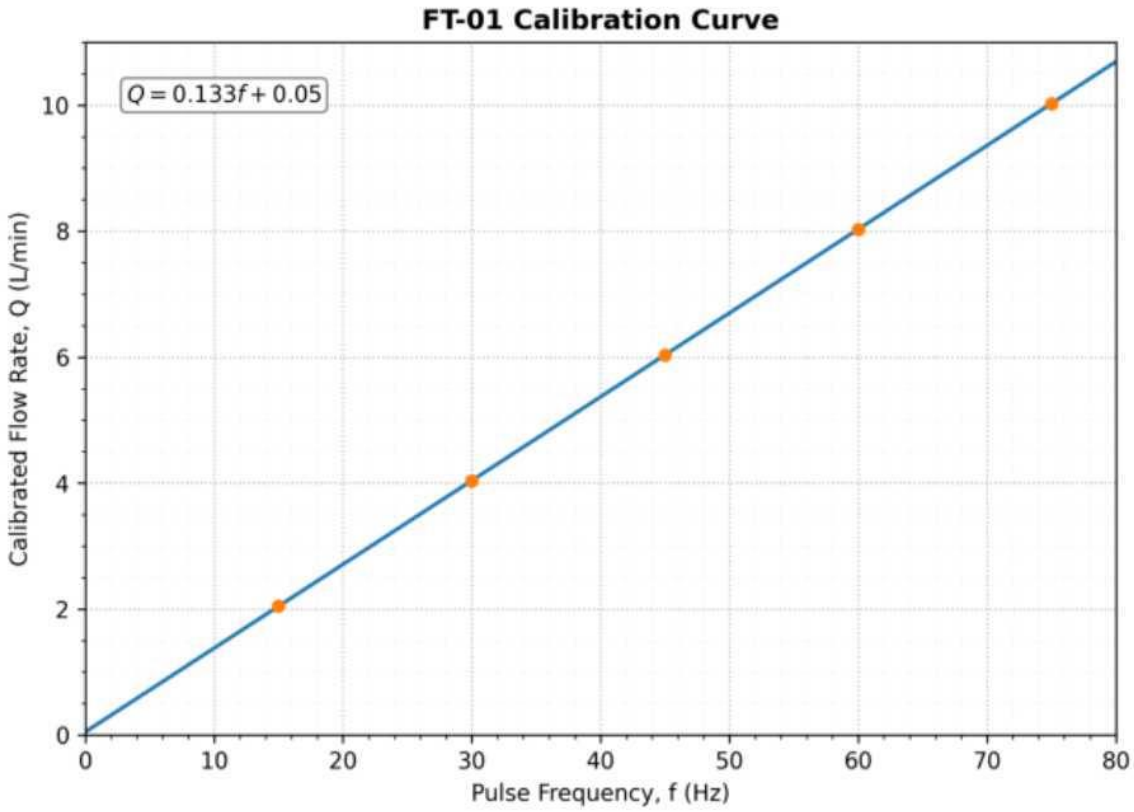
Q2.3 FT-01 produces pluses corresponding to water flow. Run the system at least for 5 different flow conditions within the available operating range (Hint: use formulas f=N/t and Q=V/t). For each condition, record the following:

Trial	Pulses	Time(s)	Frequency (Hz)	Collected Volume (L)	Actual Flow (L/min)
1	150	10	15.0	0.16	1.0
2	300	10	30.0	0.33	2.0
3	450	10	45.0	0.50	3.0
4	600	10	60.0	0.67	4.0
5	750	10	75.0	0.83	5.0

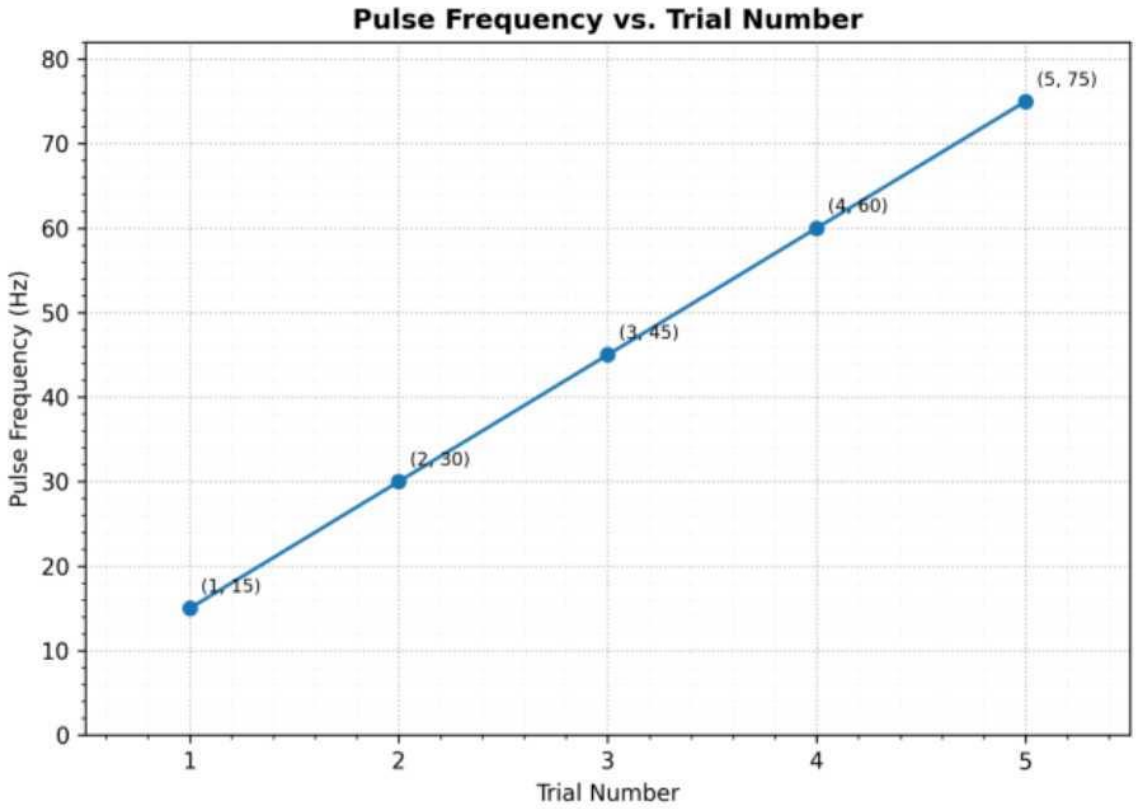
GRAPH: (2.1)



GRAPH: (2.2)


FT-01 Calibration Curve
$Q = 0.133f + 0.05$
Calibrated Flow Rate, Q (L/min)
Pulse Frequency, f (Hz)
0
2
4
6
8
10
0
10
20
30
40
50
60
70
80

GRPAH: (2.3)


Pulse Frequency vs. Trial Number
Pulse Frequency (Hz)
Trial Number
0
10
20
30
40
50
60
70
80
1
2
3
4
5
(1, 15)
(2, 30)
(3, 45)
(4, 60)
(5, 75)

$$f = \frac{N}{t}$$

Trial 1 :-

$$f = \frac{150}{10} = 15 \text{ Hz}$$

$$Q = \frac{60V}{t} = \frac{60(0.16)}{10} = 0.96 \text{ L/min} \approx 1.0 \text{ L/min}$$

Pulse frequency increases with the number of pulses counted during the fixed 10-second interval. The collected volume and actual flow rate increases across the five trials.

Graph-1: calibration curve - X-axis :-

Pulse frequency (Hz) ; Y-axis : Flow rate (L/min).

Use the supplied equation.

$$Q = 0.133f + 0.05$$

Graph-2: Pulse frequency vs trial number - X-axis :-

Trial number ; Y-axis : Pulse frequency (Hz).

Plot (1,15), (2,30), (3,45), (4,60) and (5,75).

TASK 3: HYDRAULIC SYSTEM THEORY AND FAULT ANALYSIS

Q3.1 The effective liquid volume inside a section of tubing is 0.5L, while the operating flow rate is 2.5L/min. The hydraulic residence time is given by $T_r = Q/V$. Calculate the residence time in seconds, also answer the following questions:

1. What does residence time physically represent in this system?
2. What happens to residence time if the tubing volume is doubled while the flow rate remains constant?
3. What happens to residence time if the flow rate is doubled while the tubing volume remains constant?
4. Why is residence time important when interpreting a sudden change in measured flow?

ANSWER

1. Residence time:

Given $V = 0.5\ L$ and $Q = 2.5\ L/min$

$$T_r = \frac{V}{Q} = \frac{0.5}{2.5} = 0.2\ min$$

$$T_r = 12\ seconds$$

2. If tubing volume is doubled:

Residence time doubles to 24 seconds.

3. If flow rate is doubled:

Residence time halves to 6 seconds.

4. Why is residence time important?

Residence time represents the average time taken by the liquid to pass through a tubing section. It helps to interpret the delay between a change in flow and the corresponding measurement, preventing normal transport delay from being mistaken for a fault.

Q3.2 The following observation are obtained from the same system:

CONDITION	PUMP	VALVE	FT-01 PULSE SIGNAL	MEASURED FLOW
A	ON	Open	Normal	Normal
B	ON	Partially closed	Reduced	Reduced
C	ON	Open	Zero	Zero
D	OFF	Open	Zero	Zero
E	ON	Open	Intermittent	Fluctuating

For each of the condition, identify the most likely physical or instrumentation condition. Then answer: Which condition is most difficult to distinguish using FT-01 alone, and what additional information would you require to distinguish the fault? (Hint: Teams must consider possibilities such as: Valve restriction, pump failure, hydraulic blockage, sensor/signal failure, unstable flow etc)

ANSWER

Condition	Most likely condition
A	Normal operation; pump and valve are operating correctly.
B	Valve restriction due to partial closure, reducing flow.
C	Possible hydraulic blockage or flow-sensor/signal failure.
D	Pump is OFF, resulting in zero flow.
E	Unstable flow or intermittent sensor signal, potentially due to air, fluctuating flow or a loose connection.

Most difficult condition to diagnose using FT-01 alone: condition C may indicate a pipe blockage or a faulty flow sensor, even though the pump is ON & valve is open. check pump status, sensor wiring & actual flow.

HARDWARE LIST

Mechanical Hardware

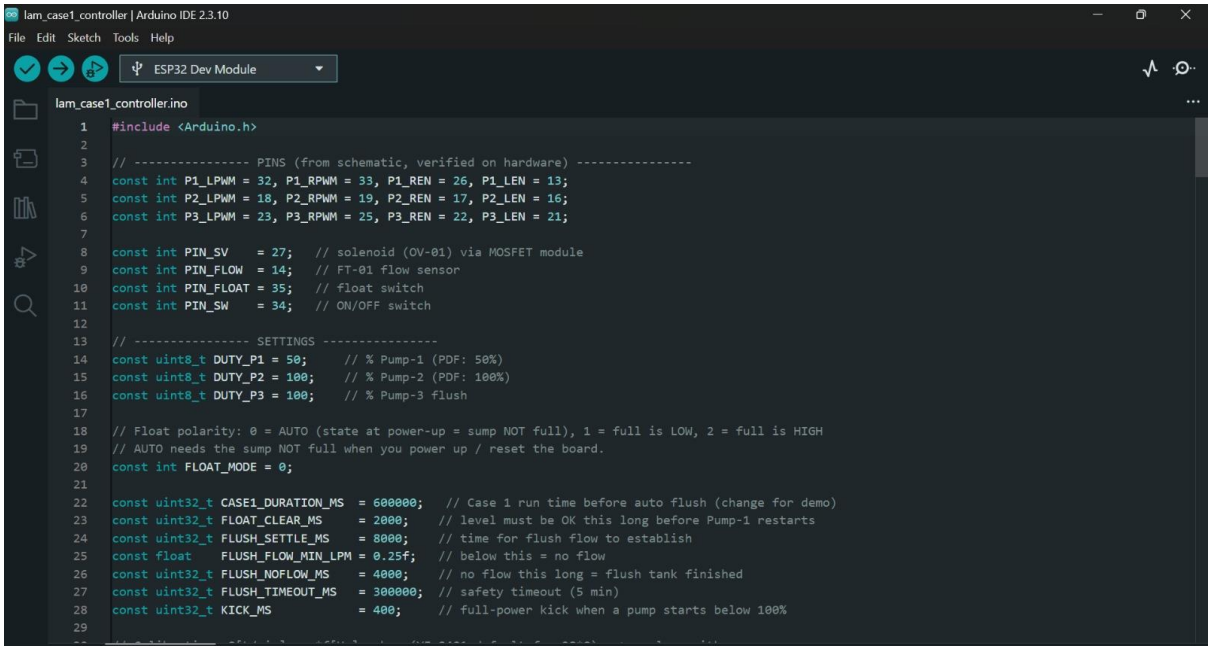
Component	Description	Main Use
Ball Valve (BV-01)	A manually operated valve with a spherical shut-off mechanism.	Used to **manually isolate or permit fluid flow** in the pipeline. It is useful for maintenance, testing, and flow isolation.
10-Litre Fluid Tank (Fluid Tank-01)	Transparent storage tank with approximately 10 L capacity.	Used as the **main fluid reservoir** for storing process fluid. The transparent body also allows the fluid level and condition to be visually monitored.
5-Litre Sump	Small transparent reservoir labelled **SUMP**, approximately 4.5 L.	Used as a **collection/storage vessel** for fluid returning to or entering the sump section of the system.
4.2-Litre Flush Fluid Tank / Flush Line	The transparent vessel labelled **FLUSH FLUID** is approximately 4.5 L.	Provides **flush fluid** for cleaning or flushing the main pipe circuit when required.
ON/OFF Valve (OV-01)	Electrically operated valve/solenoid-type valve shown in the setup.	Used for **automatic control of fluid flow**, allowing the controller to turn a particular flow path on or off.
Pumps -Pump-1, Pump-2, Pump-3	Motor-driven fluid pumps. Three pumps are shown in the setup.	Used to **circulate and transfer fluid** between different sections of the system. Each pump can be independently controlled.
8 mm Process Pipe	Flexible blue tubing with approximately 8 mm process-line size.	Used as the **main fluid-flow path** connecting tanks, valves, pumps, tees, and other components.
10 mm × 8 mm Reducers	Reducer fittings that transition between a 10 mm connection and an 8 mm pipe.	Used to **adapt the pump connection size to the 8 mm process pipe. Mainly used for Pumps**
8 mm Connectors	Push-fit/barbed-type fittings compatible with the 8 mm tubing.	Used to **join pipes to components** and to make detachable connections between sections of tubing.
8 mm Equal Tee	T-shaped connector having three equal 8 mm ports.	Used to **split one 8 mm fluid line into two branches or combine two branches into one line.**

Electronic Hardware

Electronic Component	Identification / Function	Use
PCB -Printed Circuit Board	Custom control board	Provides the electrical interconnection between the **12 V power supply, ESP32 controller, sensors, valves, indicators, and pump-driver interfaces.**
ESP32 / NodeMCU ESP32S module (U7)	Main microcontroller	Acts as the **central controller.** It provides GPIO/PWM signals for controlling outputs and receiving signals from sensors/switches.

Electronic Component	Identification / Function	Use
12 V DC Input Connector	Marked **12V INPUT**	Connection point for the external 12 V DC power supply.
12 V Output Connectors	P1, P2, P3 and additional 12 V output terminals	Provide 12 V power connections for external loads such as the pump-driver/motor circuits.
Motor Driver Interfaces -U2, U3, U4	Three multi-pin driver connectors shown on the PCB	Provide dedicated interfaces for **Pump-1, Pump-2 and Pump-3 driver/control connections**.
Solenoid Valve Connector	Labelled **SOLENOID_VALVE -S/V/G**	Interface for controlling/connecting the solenoid or ON/OFF valve.
Flow Sensor Connector	Labelled **FLOW_SENSOR - S/V/G**	Interface for the water-flow sensor; supplies power and receives the sensor signal. **THE FLOW SENSOR SHOULD BE CALIBRATED**
Float Switch Connector	Labelled **FLOAT_SWITCH -FSW/GND**	Interface for a float/level switch used to detect a water-level condition.
ON/OFF Switch Connector	Labelled **ON/OFF_SWITCH -SW/GND**	Interface for an external manual ON/OFF switch.
12V DC SMPS	12V DC SMPS	For Power Supply to the board

CODE:




```ino
#include <Arduino.h>

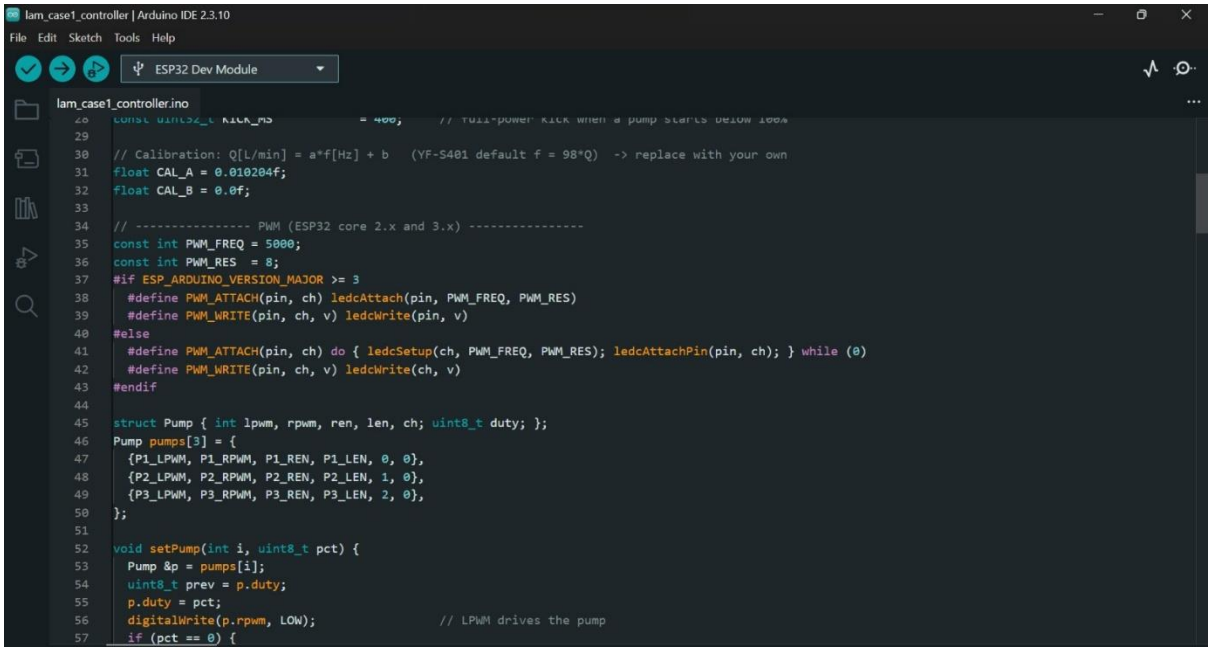
// ---------------- PINS (from schematic, verified on hardware) ----------------
const int P1_LPWM = 32, P1_RPWM = 33, P1_REN = 26, P1_LEN = 13;
const int P2_LPWM = 18, P2_RPWM = 19, P2_REN = 17, P2_LEN = 16;
const int P3_LPWM = 23, P3_RPWM = 25, P3_REN = 22, P3_LEN = 21;

const int PIN_SV    = 27;   // solenoid (OV-01) via MOSFET module
const int PIN_FLOW  = 14;   // FT-01 flow sensor
const int PIN_FLOAT = 35;   // float switch
const int PIN_SW    = 34;   // ON/OFF switch

// ---------------- SETTINGS ----------------
const uint8_t DUTY_P1 = 50;     // % Pump-1 (PDF: 50%)
const uint8_t DUTY_P2 = 100;    // % Pump-2 (PDF: 100%)
const uint8_t DUTY_P3 = 100;    // % Pump-3 flush

// Float polarity: 0 = AUTO (state at power-up = sump NOT full), 1 = full is LOW, 2 = full is HIGH
// AUTO needs the sump NOT full when you power up / reset the board.
const int FLOAT_MODE = 0;

const uint32_t CASE1_DURATION_MS  = 600000;   // Case 1 run time before auto flush (change for demo)
const uint32_t FLOAT_CLEAR_MS     = 2000;     // level must be OK this long before Pump-1 restarts
const uint32_t FLUSH_SETTLE_MS    = 8000;     // time for flush flow to establish
const float    FLUSH_FLOW_MIN_LPM = 0.25f;    // below this = no flow
const uint32_t FLUSH_NOFLOW_MS    = 4000;     // no flow this long = flush tank finished
const uint32_t FLUSH_TIMEOUT_MS   = 300000;   // safety timeout (5 min)
const uint32_t KICK_MS            = 400;      // full-power kick when a pump starts below 100%
```

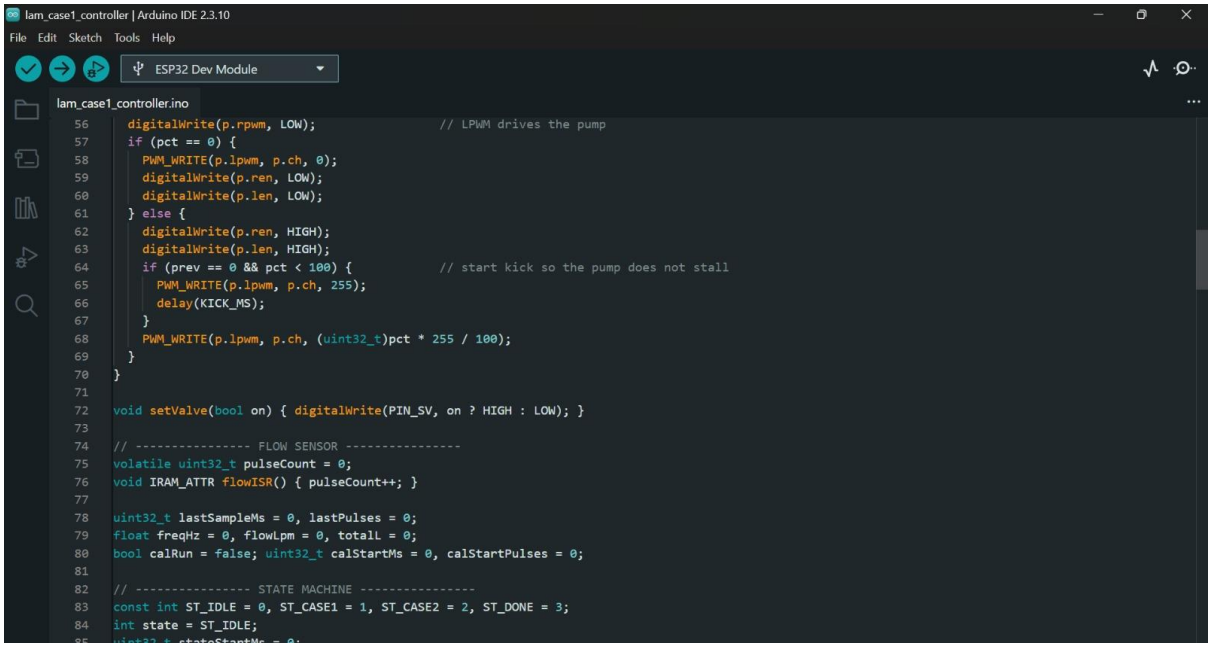


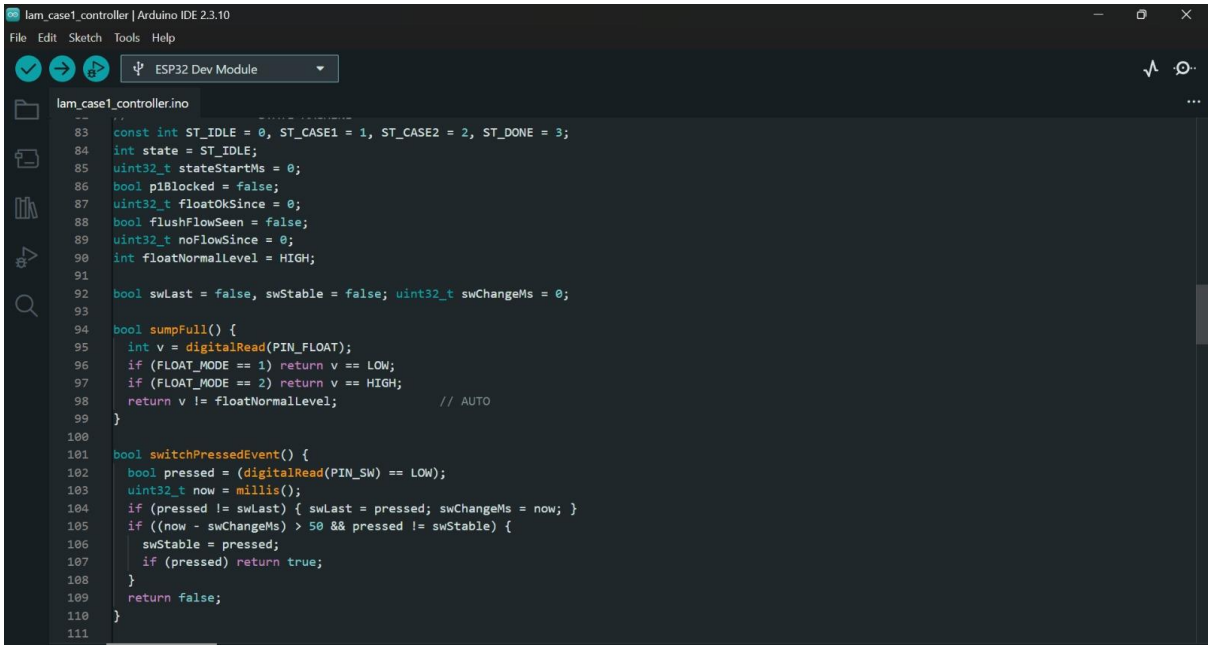

```ino
// Calibration: Q[L/min] = a*f[Hz] + b   (YF-S401 default f = 98*Q)  -> replace with your own
float CAL_A = 0.010204f;
float CAL_B = 0.0f;

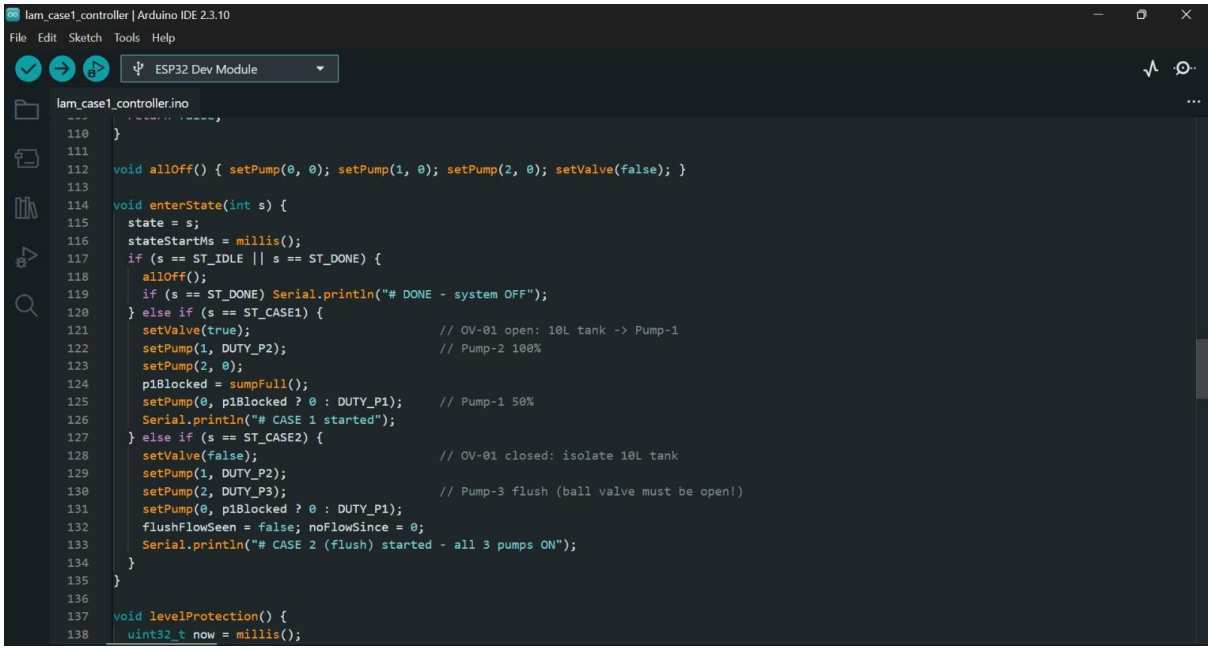
// ---------------- PWM (ESP32 core 2.x and 3.x) ----------------
const int PWM_FREQ = 5000;
const int PWM_RES  = 8;
#if ESP_ARDUINO_VERSION_MAJOR >= 3
  #define PWM_ATTACH(pin, ch) ledcAttach(pin, PWM_FREQ, PWM_RES)
  #define PWM_WRITE(pin, ch, v) ledcWrite(pin, v)
#else
  #define PWM_ATTACH(pin, ch) do { ledcSetup(ch, PWM_FREQ, PWM_RES); ledcAttachPin(pin, ch); } while (0)
  #define PWM_WRITE(pin, ch, v) ledcWrite(ch, v)
#endif

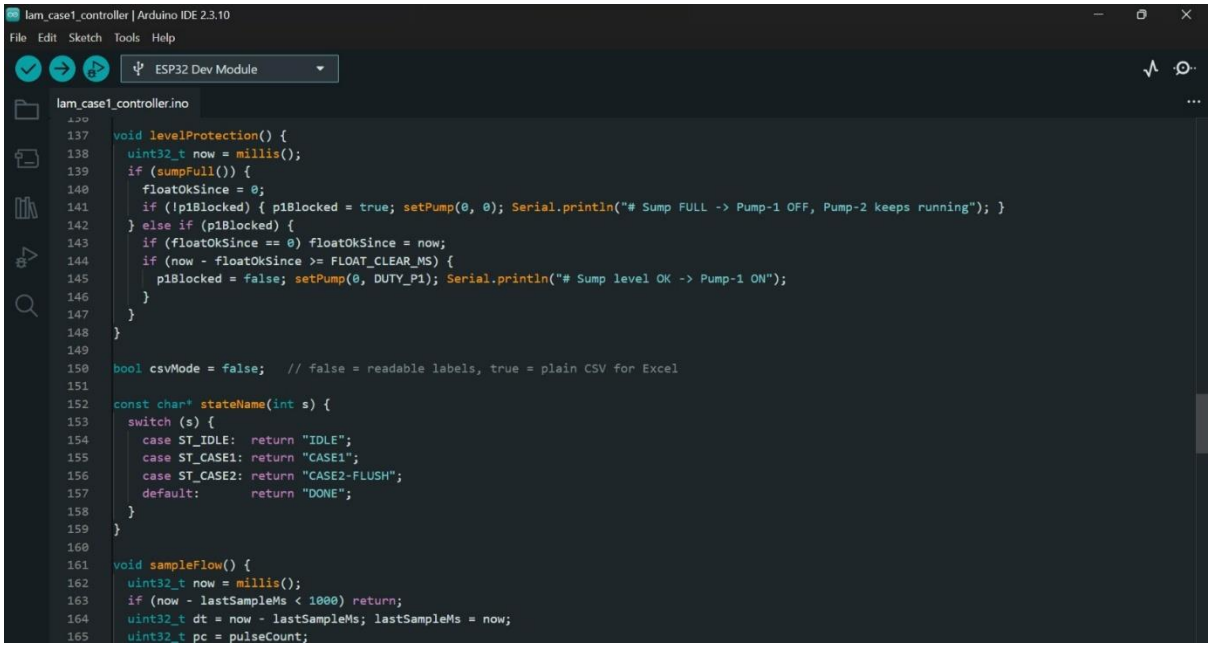
struct Pump { int lpwm, rpwm, ren, len, ch; uint8_t duty; };
Pump pumps[3] = {
  {P1_LPWM, P1_RPWM, P1_REN, P1_LEN, 0, 0},
  {P2_LPWM, P2_RPWM, P2_REN, P2_LEN, 1, 0},
  {P3_LPWM, P3_RPWM, P3_REN, P3_LEN, 2, 0},
};

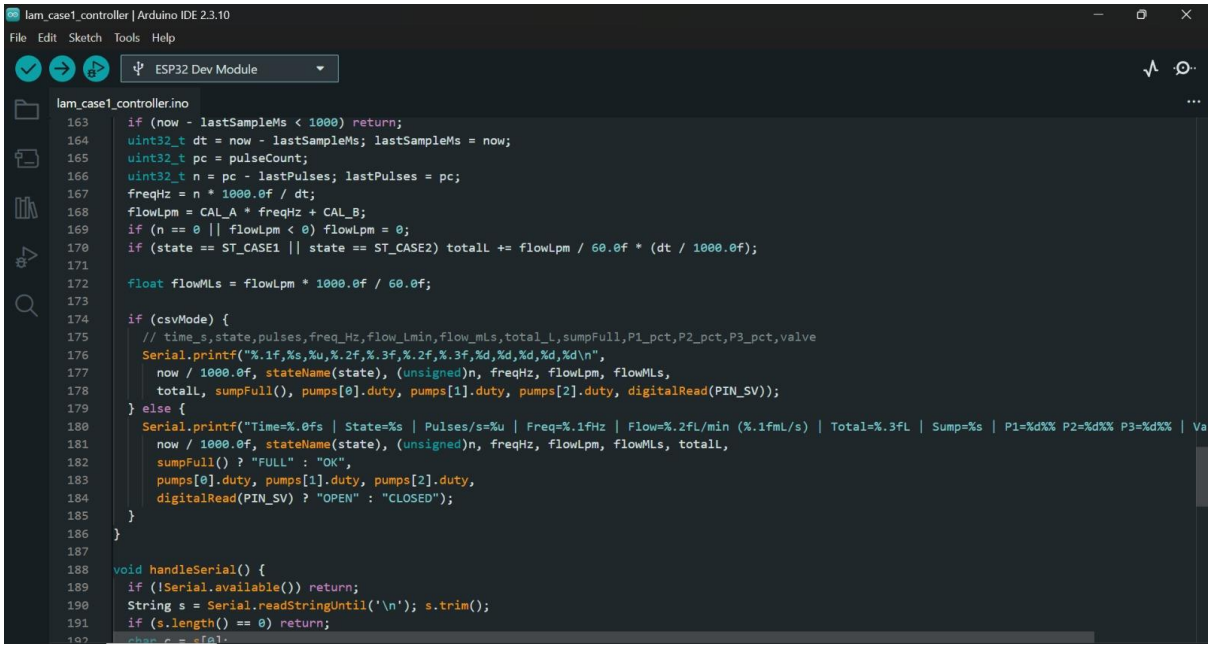
void setPump(int i, uint8_t pct) {
  Pump &p = pumps[i];
  uint8_t prev = p.duty;
  p.duty = pct;
  digitalWrite(p.rpwm, LOW);                // LPWM drives the pump
  if (pct == 0) {
```

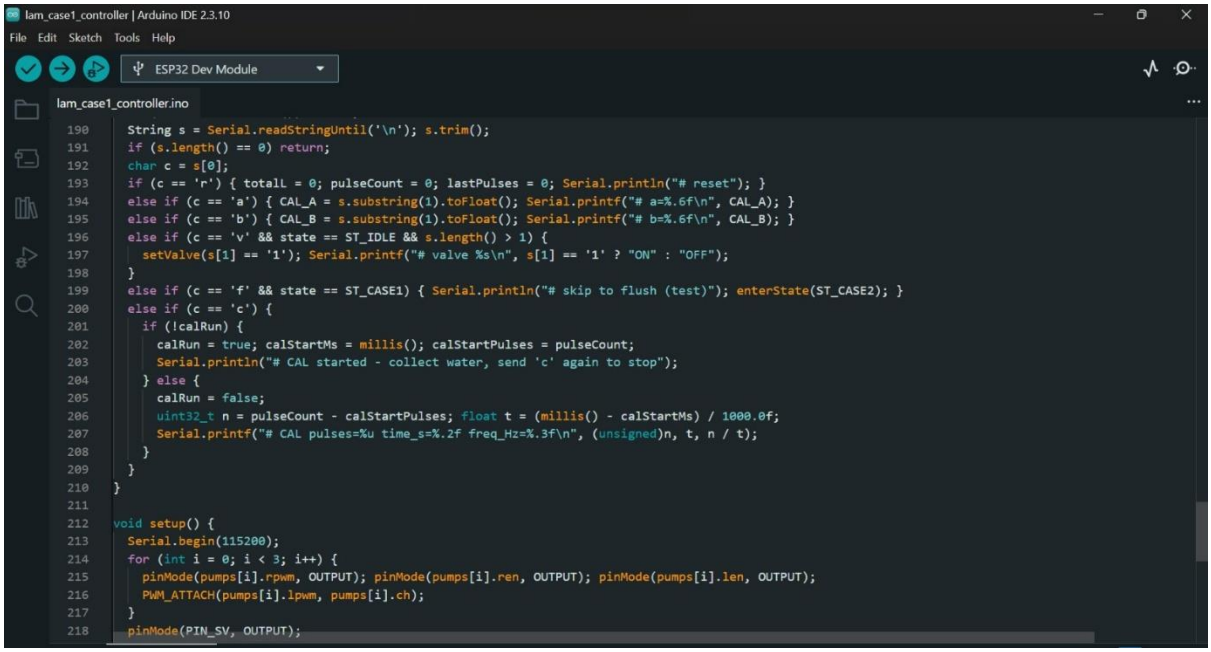

lam_case1_controller | Arduino IDE 2.3.10
File Edit Sketch Tools Help
ESP32 Dev Module
lam_case1_controller.ino
digitalWrite(p.rpwm, LOW); // LPWM drives the pump
if (pct == 0) {
PWM_WRITE(p.lpwm, p.ch, 0);
digitalWrite(p.ren, LOW);
digitalWrite(p.len, LOW);
} else {
digitalWrite(p.ren, HIGH);
digitalWrite(p.len, HIGH);
if (prev == 0 && pct < 100) { // start kick so the pump does not stall
PWM_WRITE(p.lpwm, p.ch, 255);
delay(KICK_MS);
}
PWM_WRITE(p.lpwm, p.ch, (uint32_t)pct * 255 / 100);
}
}
void setValve(bool on) { digitalWrite(PIN_SV, on ? HIGH : LOW); }
// ---------- FLOW SENSOR ----------
volatile uint32_t pulseCount = 0;
void IRAM_ATTR flowISR() { pulseCount++; }
uint32_t lastSampleMs = 0, lastPulses = 0;
float freqHz = 0, flowLpm = 0, totalL = 0;
bool calRun = false; uint32_t calStartMs = 0, calStartPulses = 0;
// ---------- STATE MACHINE ----------
const int ST_IDLE = 0, ST_CASE1 = 1, ST_CASE2 = 2, ST_DONE = 3;
int state = ST_IDLE;

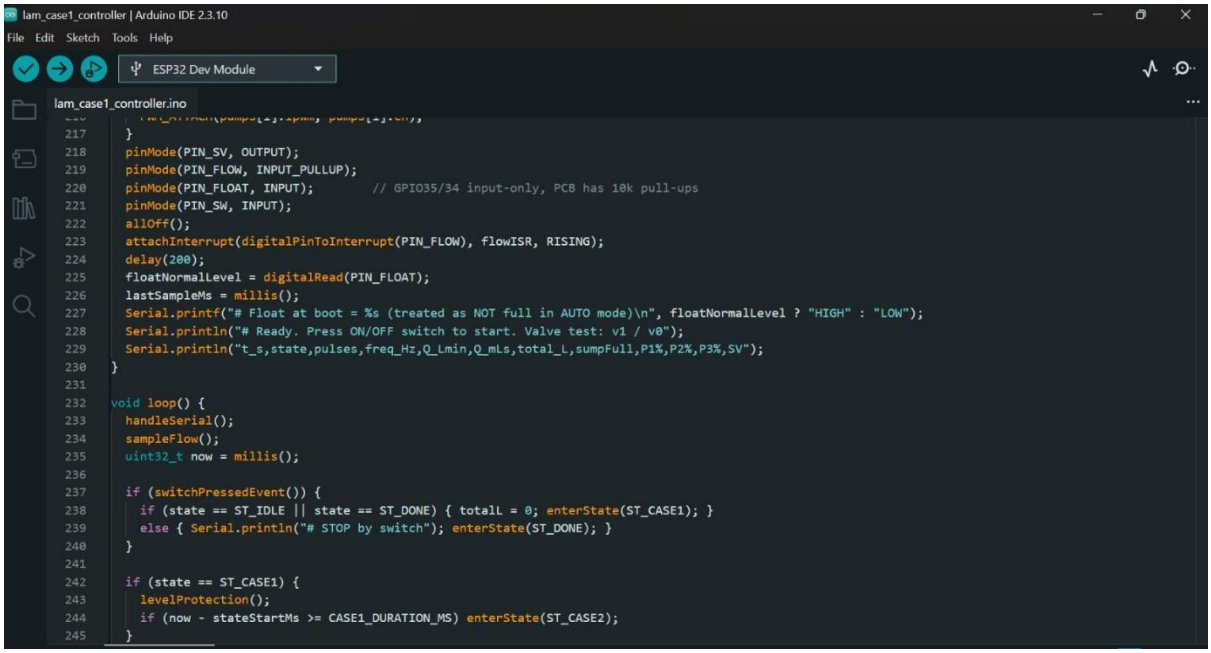

lam_case1_controller | Arduino IDE 2.3.10
File Edit Sketch Tools Help
ESP32 Dev Module
lam_case1_controller.ino
const int ST_IDLE = 0, ST_CASE1 = 1, ST_CASE2 = 2, ST_DONE = 3;
int state = ST_IDLE;
uint32_t stateStartMs = 0;
bool p1Blocked = false;
uint32_t floatOkSince = 0;
bool flushFlowSeen = false;
uint32_t noFlowSince = 0;
int floatNormalLevel = HIGH;
bool swLast = false, swStable = false; uint32_t swChangeMs = 0;
bool sumpFull() {
int v = digitalRead(PIN_FLOAT);
if (FLOAT_MODE == 1) return v == LOW;
if (FLOAT_MODE == 2) return v == HIGH;
return v != floatNormalLevel; // AUTO
}
bool switchPressedEvent() {
bool pressed = (digitalRead(PIN_SW) == LOW);
uint32_t now = millis();
if (pressed != swLast) { swLast = pressed; swChangeMs = now; }
if ((now - swChangeMs) > 50 && pressed != swStable) {
swStable = pressed;
if (pressed) return true;
}
return false;
}

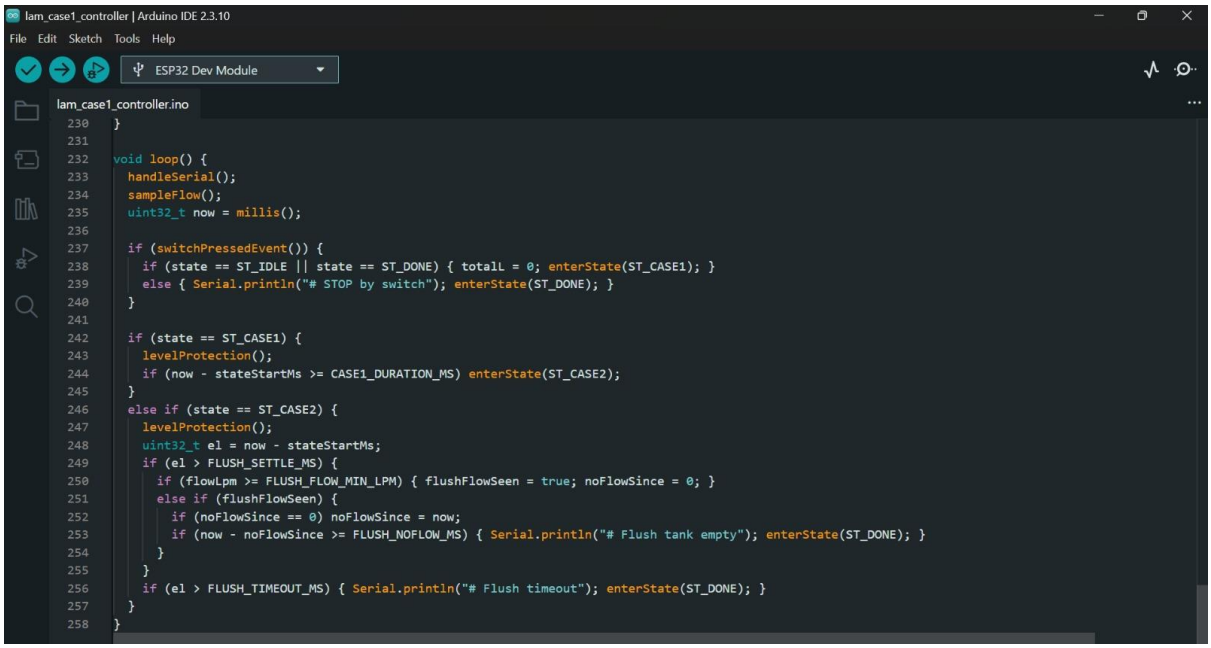

lam_case1_controller | Arduino IDE 2.3.10
File Edit Sketch Tools Help
ESP32 Dev Module
lam_case1_controller.ino
}
void allOff() { setPump(0, 0); setPump(1, 0); setPump(2, 0); setValve(false); }
void enterState(int s) {
state = s;
stateStartMs = millis();
if (s == ST_IDLE || s == ST_DONE) {
allOff();
if (s == ST_DONE) Serial.println("# DONE - system OFF");
} else if (s == ST_CASE1) {
setValve(true); // OV-01 open: 10L tank -> Pump-1
setPump(1, DUTY_P2); // Pump-2 100%
setPump(2, 0);
p1Blocked = sumpFull();
setPump(0, p1Blocked ? 0 : DUTY_P1); // Pump-1 50%
Serial.println("# CASE 1 started");
} else if (s == ST_CASE2) {
setValve(false); // OV-01 closed: isolate 10L tank
setPump(1, DUTY_P2);
setPump(2, DUTY_P3); // Pump-3 flush (ball valve must be open!)
setPump(0, p1Blocked ? 0 : DUTY_P1);
flushFlowSeen = false; noFlowSince = 0;
Serial.println("# CASE 2 (flush) started - all 3 pumps ON");
}
}
void levelProtection() {
uint32_t now = millis();


lam_case1_controller | Arduino IDE 2.3.10
File Edit Sketch Tools Help
ESP32 Dev Module
lam_case1_controller.ino
void levelProtection() {
uint32_t now = millis();
if (sumpFull()) {
floatOkSince = 0;
if (!p1Blocked) { p1Blocked = true; setPump(0, 0); Serial.println("# Sump FULL -> Pump-1 OFF, Pump-2 keeps running"); }
} else if (p1Blocked) {
if (floatOkSince == 0) floatOkSince = now;
if (now - floatOkSince >= FLOAT_CLEAR_MS) {
p1Blocked = false; setPump(0, DUTY_P1); Serial.println("# Sump level OK -> Pump-1 ON");
}
}
}
bool csvMode = false; // false = readable labels, true = plain CSV for Excel
const char* stateName(int s) {
switch (s) {
case ST_IDLE: return "IDLE";
case ST_CASE1: return "CASE1";
case ST_CASE2: return "CASE2-FLUSH";
default: return "DONE";
}
}
void sampleFlow() {
uint32_t now = millis();
if (now - lastSampleMs < 1000) return;
uint32_t dt = now - lastSampleMs; lastSampleMs = now;
uint32_t pc = pulseCount;


lam_case1_controller | Arduino IDE 2.3.10
File Edit Sketch Tools Help
ESP32 Dev Module
lam_case1_controller.ino
if (now - lastSampleMs < 1000) return;
uint32_t dt = now - lastSampleMs; lastSampleMs = now;
uint32_t pc = pulseCount;
uint32_t n = pc - lastPulses; lastPulses = pc;
freqHz = n * 1000.0f / dt;
flowLpm = CAL_A * freqHz + CAL_B;
if (n == 0 || flowLpm < 0) flowLpm = 0;
if (state == ST_CASE1 || state == ST_CASE2) totalL += flowLpm / 60.0f * (dt / 1000.0f);
float flowMLs = flowLpm * 1000.0f / 60.0f;
if (csvMode) {
// time_s,state,pulses,freq_Hz,flow_Lmin,flow_mLs,total_L,sumpFull,P1_pct,P2_pct,P3_pct,valve
Serial.printf("%.1f,%s,%u,%.2f,%.3f,%.2f,%.3f,%d,%d,%d,%d,%d\n",
now / 1000.0f, stateName(state), (unsigned)n, freqHz, flowLpm, flowMLs,
totalL, sumpFull(), pumps[0].duty, pumps[1].duty, pumps[2].duty, digitalRead(PIN_SV));
} else {
Serial.printf("Time=%.0fs | State=%s | Pulses/s=%u | Freq=%.1fHz | Flow=%.2fL/min (%.1fmL/s) | Total=%.3fL | Sump=%s | P1=%d%% P2=%d%% P3=%d%% | Va
now / 1000.0f, stateName(state), (unsigned)n, freqHz, flowLpm, flowMLs, totalL,
sumpFull() ? "FULL" : "OK",
pumps[0].duty, pumps[1].duty, pumps[2].duty,
digitalRead(PIN_SV) ? "OPEN" : "CLOSED");
}
}
void handleSerial() {
if (!Serial.available()) return;
String s = Serial.readStringUntil('\n'); s.trim();
if (s.length() == 0) return;


lam_case1_controller | Arduino IDE 2.3.10
File Edit Sketch Tools Help
ESP32 Dev Module
lam_case1_controller.ino
String s = Serial.readStringUntil('\n'); s.trim();
if (s.length() == 0) return;
char c = s[0];
if (c == 'r') { totalL = 0; pulseCount = 0; lastPulses = 0; Serial.println("# reset"); }
else if (c == 'a') { CAL_A = s.substring(1).toFloat(); Serial.printf("# a=%.6f\n", CAL_A); }
else if (c == 'b') { CAL_B = s.substring(1).toFloat(); Serial.printf("# b=%.6f\n", CAL_B); }
else if (c == 'v' && state == ST_IDLE && s.length() > 1) {
setValve(s[1] == '1'); Serial.printf("# valve %s\n", s[1] == '1' ? "ON" : "OFF");
}
else if (c == 'f' && state == ST_CASE1) { Serial.println("# skip to flush (test)"); enterState(ST_CASE2); }
else if (c == 'c') {
if (!calRun) {
calRun = true; calStartMs = millis(); calStartPulses = pulseCount;
Serial.println("# CAL started - collect water, send 'c' again to stop");
} else {
calRun = false;
uint32_t n = pulseCount - calStartPulses; float t = (millis() - calStartMs) / 1000.0f;
Serial.printf("# CAL pulses=%u time_s=%.2f freq_Hz=%.3f\n", (unsigned)n, t, n / t);
}
}
}
void setup() {
Serial.begin(115200);
for (int i = 0; i < 3; i++) {
pinMode(pumps[i].rpwm, OUTPUT); pinMode(pumps[i].ren, OUTPUT); pinMode(pumps[i].len, OUTPUT);
PWM_ATTACH(pumps[i].lpwm, pumps[i].ch);
}
pinMode(PIN_SV, OUTPUT);





CODE IN TEXT:

```
#include <Arduino.h>

const int P1_LPWM = 32, P1_RPWM = 33, P1_REN = 26, P1_LEN = 13;
const int P2_LPWM = 18, P2_RPWM = 19, P2_REN = 17, P2_LEN = 16;
const int P3_LPWM = 23, P3_RPWM = 25, P3_REN = 22, P3_LEN = 21;
```

```
const int PIN_SV    = 27;
const int PIN_FLOW  = 14;
const int PIN_FLOAT = 35;
const int PIN_SW    = 34;

const uint8_t DUTY_P1 = 50;
const uint8_t DUTY_P2 = 100;
const uint8_t DUTY_P3 = 100;

const int FLOAT_MODE = 0;

const uint32_t CASE1_DURATION_MS  = 600000;
const uint32_t FLOAT_CLEAR_MS     = 2000;
const uint32_t FLUSH_SETTLE_MS    = 8000;
const float    FLUSH_FLOW_MIN_LPM = 0.25f;
const uint32_t FLUSH_NOFLOW_MS    = 4000;
const uint32_t FLUSH_TIMEOUT_MS   = 300000;
const uint32_t KICK_MS            = 400;

float CAL_A = 0.010204f;
float CAL_B = 0.0f;

const int PWM_FREQ = 5000;
const int PWM_RES  = 8;
```

```
#if ESP_ARDUINO_VERSION_MAJOR >= 3
  #define PWM_ATTACH(pin, ch) ledcAttach(pin, PWM_FREQ, PWM_RES)
  #define PWM_WRITE(pin, ch, v) ledcWrite(pin, v)
#else
  #define PWM_ATTACH(pin, ch) do { ledcSetup(ch, PWM_FREQ, PWM_RES); ledcAttachPin(pin, ch); } while (0)
  #define PWM_WRITE(pin, ch, v) ledcWrite(ch, v)
#endif

struct Pump { int lpwm, rpwm, ren, len, ch; uint8_t duty; };
Pump pumps[3] = {
  {P1_LPWM, P1_RPWM, P1_REN, P1_LEN, 0, 0},
  {P2_LPWM, P2_RPWM, P2_REN, P2_LEN, 1, 0},
  {P3_LPWM, P3_RPWM, P3_REN, P3_LEN, 2, 0},
};

void setPump(int i, uint8_t pct) {
  Pump &p = pumps[i];
  uint8_t prev = p.duty;
  p.duty = pct;
  digitalWrite(p.rpwm, LOW);
  if (pct == 0) {
    PWM_WRITE(p.lpwm, p.ch, 0);
    digitalWrite(p.ren, LOW);
    digitalWrite(p.len, LOW);
```

```
  } else {
    digitalWrite(p.ren, HIGH);
    digitalWrite(p.len, HIGH);
    if (prev == 0 && pct < 100) {
      PWM_WRITE(p.lpwm, p.ch, 255);
      delay(KICK_MS);
    }
    PWM_WRITE(p.lpwm, p.ch, (uint32_t)pct * 255 / 100);
  }
}

void setValve(bool on) { digitalWrite(PIN_SV, on ? HIGH : LOW); }

volatile uint32_t pulseCount = 0;
void IRAM_ATTR flowISR() { pulseCount++; }

uint32_t lastSampleMs = 0, lastPulses = 0;
float freqHz = 0, flowLpm = 0, totalL = 0;
bool calRun = false; uint32_t calStartMs = 0, calStartPulses = 0;

const int ST_IDLE = 0, ST_CASE1 = 1, ST_CASE2 = 2, ST_DONE = 3;
int state = ST_IDLE;
uint32_t stateStartMs = 0;
bool p1Blocked = false;
uint32_t floatOkSince = 0;
```

```
bool flushFlowSeen = false;
uint32_t noFlowSince = 0;
int floatNormalLevel = HIGH;

bool swLast = false, swStable = false; uint32_t swChangeMs = 0;

bool sumpFull() {
  int v = digitalRead(PIN_FLOAT);
  if (FLOAT_MODE == 1) return v == LOW;
  if (FLOAT_MODE == 2) return v == HIGH;
  return v != floatNormalLevel;
}

bool switchPressedEvent() {
  bool pressed = (digitalRead(PIN_SW) == LOW);
  uint32_t now = millis();
  if (pressed != swLast) { swLast = pressed; swChangeMs = now; }
  if ((now - swChangeMs) > 50 && pressed != swStable) {
    swStable = pressed;
    if (pressed) return true;
  }
  return false;
}
```

```
void allOff() { setPump(0, 0); setPump(1, 0); setPump(2, 0);
setValve(false); }


void enterState(int s) {
  state = s;
  stateStartMs = millis();
  if (s == ST_IDLE || s == ST_DONE) {
    allOff();
    if (s == ST_DONE) Serial.println("# DONE - system OFF");
  } else if (s == ST_CASE1) {
    setValve(true);
    setPump(1, DUTY_P2);
    setPump(2, 0);
    p1Blocked = sumpFull();
    setPump(0, p1Blocked ? 0 : DUTY_P1);
    Serial.println("# CASE 1 started");
  } else if (s == ST_CASE2) {
    setValve(false);
    setPump(1, DUTY_P2);
    setPump(2, DUTY_P3);
    setPump(0, p1Blocked ? 0 : DUTY_P1);
    flushFlowSeen = false; noFlowSince = 0;
    Serial.println("# CASE 2 (flush) started - all 3 pumps ON");
  }
}
```

```
void levelProtection() {
  uint32_t now = millis();
  if (sumpFull()) {
    floatOkSince = 0;
    if (!p1Blocked) { p1Blocked = true; setPump(0, 0); Serial.println("# Sump FULL -> Pump-1 OFF, Pump-2 keeps running"); }
  } else if (p1Blocked) {
    if (floatOkSince == 0) floatOkSince = now;
    if (now - floatOkSince >= FLOAT_CLEAR_MS) {
      p1Blocked = false; setPump(0, DUTY_P1); Serial.println("# Sump level OK -> Pump-1 ON");
    }
  }
}

bool csvMode = false;

const char* stateName(int s) {
  switch (s) {
    case ST_IDLE:  return "IDLE";
    case ST_CASE1: return "CASE1";
    case ST_CASE2: return "CASE2-FLUSH";
    default:       return "DONE";
  }
}
```

```
void sampleFlow() {
  uint32_t now = millis();
  if (now - lastSampleMs < 1000) return;
  uint32_t dt = now - lastSampleMs; lastSampleMs = now;
  uint32_t pc = pulseCount;
  uint32_t n = pc - lastPulses; lastPulses = pc;
  freqHz = n * 1000.0f / dt;
  flowLpm = CAL_A * freqHz + CAL_B;
  if (n == 0 || flowLpm < 0) flowLpm = 0;
  if (state == ST_CASE1 || state == ST_CASE2) totalL += flowLpm / 60.0f *
(dt / 1000.0f);

  float flowMLs = flowLpm * 1000.0f / 60.0f;

  if (csvMode) {

    Serial.printf("%.1f,%s,%u,%.2f,%.3f,%.2f,%.3f,%d,%d,%d,%d,%d\n",
      now / 1000.0f, stateName(state), (unsigned)n, freqHz, flowLpm,
flowMLs,
      totalL, sumpFull(), pumps[0].duty, pumps[1].duty, pumps[2].duty,
digitalRead(PIN_SV));
  } else {
    Serial.printf("Time=%.0fs | State=%s | Pulses/s=%u | Freq=%.1fHz |
Flow=%.2fL/min (%.1fmL/s) | Total=%.3fL | Sump=%s | P1=%d%%
P2=%d%% P3=%d%% | Valve=%s\n",
```

```
    now / 1000.0f, stateName(state), (unsigned)n, freqHz, flowLpm,
flowMLs, totalL,
    sumpFull() ? "FULL" : "OK",
    pumps[0].duty, pumps[1].duty, pumps[2].duty,
    digitalRead(PIN_SV) ? "OPEN" : "CLOSED");
  }
}

void handleSerial() {
  if (!Serial.available()) return;
  String s = Serial.readStringUntil('\n'); s.trim();
  if (s.length() == 0) return;
  char c = s[0];
  if (c == 'r') { totalL = 0; pulseCount = 0; lastPulses = 0; Serial.println("#
reset"); }
  else if (c == 'a') { CAL_A = s.substring(1).toFloat(); Serial.printf("#
a=%.6f\n", CAL_A); }
  else if (c == 'b') { CAL_B = s.substring(1).toFloat(); Serial.printf("#
b=%.6f\n", CAL_B); }
  else if (c == 'v' && state == ST_IDLE && s.length() > 1) {
    setValve(s[1] == '1'); Serial.printf("# valve %s\n", s[1] == '1' ? "ON" :
"OFF");
  }
  else if (c == 'f' && state == ST_CASE1) { Serial.println("# skip to flush
(test)"); enterState(ST_CASE2); }
  else if (c == 'c') {
    if (!calRun) {
```

```
      calRun = true; calStartMs = millis(); calStartPulses = pulseCount;
      Serial.println("# CAL started - collect water, send 'c' again to stop");
    } else {
      calRun = false;
      uint32_t n = pulseCount - calStartPulses; float t = (millis() - calStartMs)
/ 1000.0f;
      Serial.printf("# CAL pulses=%u time_s=%.2f freq_Hz=%.3f\n",
(unsigned)n, t, n / t);
    }
  }
}

void setup() {
  Serial.begin(115200);
  for (int i = 0; i < 3; i++) {
    pinMode(pumps[i].rpwm, OUTPUT); pinMode(pumps[i].ren, OUTPUT);
pinMode(pumps[i].len, OUTPUT);
    PWM_ATTACH(pumps[i].lpwm, pumps[i].ch);
  }
  pinMode(PIN_SV, OUTPUT);
  pinMode(PIN_FLOW, INPUT_PULLUP);
  pinMode(PIN_FLOAT, INPUT);
  pinMode(PIN_SW, INPUT);
  allOff();
  attachInterrupt(digitalPinToInterrupt(PIN_FLOW), flowISR, RISING);
  delay(200);
```

```
  floatNormalLevel = digitalRead(PIN_FLOAT);
  lastSampleMs = millis();
  Serial.printf("# Float at boot = %s (treated as NOT full in AUTO mode)\n",
floatNormalLevel ? "HIGH" : "LOW");
  Serial.println("# Ready. Press ON/OFF switch to start. Valve test: v1 /
v0");
  Serial.println("t_s,state,pulses,freq_Hz,Q_Lmin,Q_mLs,total_L,sumpFull,
P1%,P2%,P3%,SV");
}

void loop() {
  handleSerial();
  sampleFlow();
  uint32_t now = millis();

  if (switchPressedEvent()) {
    if (state == ST_IDLE || state == ST_DONE) { totalL = 0;
enterState(ST_CASE1); }
    else { Serial.println("# STOP by switch"); enterState(ST_DONE); }
  }

  if (state == ST_CASE1) {
    levelProtection();
    if (now - stateStartMs >= CASE1_DURATION_MS) enterState(ST_CASE2);
  }
  else if (state == ST_CASE2) {
```

```
   levelProtection();
   uint32_t el = now - stateStartMs;
   if (el > FLUSH_SETTLE_MS) {
     if (flowLpm >= FLUSH_FLOW_MIN_LPM) { flushFlowSeen = true; noFlowSince = 0; }
     else if (flushFlowSeen) {
       if (noFlowSince == 0) noFlowSince = now;
       if (now - noFlowSince >= FLUSH_NOFLOW_MS) { Serial.println("# Flush tank empty"); enterState(ST_DONE); }
     }
   }
   if (el > FLUSH_TIMEOUT_MS) { Serial.println("# Flush timeout"); enterState(ST_DONE); }
  }
}
```